

WEEK10-PRACTICAL

Task1:

Investigate inode structures using commands such as `ls -li`, `stat`, and `find`. Create hard links and symbolic links and explain their impact on inode allocation.

```
vishalkarnati@VISHALS-MacBook-Air os % mkdir week10practical
vishalkarnati@VISHALS-MacBook-Air os % cd week10practical
vishalkarnati@VISHALS-MacBook-Air week10practical % echo "Operating Systems inode practical" > original.txt
vishalkarnati@VISHALS-MacBook-Air week10practical % cat original.txt
Operating Systems inode practical
vishalkarnati@VISHALS-MacBook-Air week10practical % ls -li original.txt
64715620 original.txt
vishalkarnati@VISHALS-MacBook-Air week10practical % stat original.txt
16777233 64715620 -rw-r--r-- 1 vishalkarnati staff 8 34 "Sep 27 11:47:08 2026" "Sep 27 11:47:07 2026" "Sep 27 11:47:07 2026" "Sep 27 11:47:07 2026" 4096 8 0 original.txt
vishalkarnati@VISHALS-MacBook-Air week10practical % ln original.txt hardlink.txt
vishalkarnati@VISHALS-MacBook-Air week10practical % ls -li original.txt hardlink.txt
64715620 -rw-r--r-- 2 vishalkarnati staff 34 27 Sep 11:47 hardlink.txt
64715620 -rw-r--r-- 2 vishalkarnati staff 34 27 Sep 11:47 original.txt
vishalkarnati@VISHALS-MacBook-Air week10practical % echo "Added through hard link" >> hardlink.txt
vishalkarnati@VISHALS-MacBook-Air week10practical % cat original.txt
Operating Systems inode practical
Added through hard link
vishalkarnati@VISHALS-MacBook-Air week10practical % ln -s original.txt symlink.txt
vishalkarnati@VISHALS-MacBook-Air week10practical % ls -li original.txt symlink.txt
ls: symlink.txt: No such file or directory
64715620 -rw-r--r-- 2 vishalkarnati staff 58 27 Sep 11:47 hardlink.txt
64715620 -rw-r--r-- 2 vishalkarnati staff 58 27 Sep 11:47 original.txt
vishalkarnati@VISHALS-MacBook-Air week10practical %
```

Task2:

Design a program using `mmap()` to perform file reading and writing. Compare memory-mapped I/O with traditional `read/write` operations in terms of efficiency and implementation complexity.

PART A — mmap() version:

```
UW PICO 5.09 File: mmap_copy.c Modified
#include <stdio.h>
#include <stdlib.h>
#include <fcntl.h>
#include <unistd.h>
#include <sys/mman.h>
#include <sys/stat.h>
#include <string.h>

int main(int argc, char *argv[])
{
    int input_fd;
    int output_fd;

    struct stat file_info;

    char *source;
    char *destination;

    if (argc != 3)
    {
        printf("Usage: %s <source> <destination>\n", argv[0]);
        return 1;
    }

    /*
     * Open source file.
     */
    input_fd = open(argv[1], O_RDONLY);

    if (input_fd == -1)
    {
        perror("open source");
        return 1;
    }

    /*
     * Get file size.
     */
    if (fstat(input_fd, &file_info) == -1)
    {
        perror("fstat");
        close(input_fd);
        return 1;
    }

    size_t size = file_info.st_size;

    if (size == 0)
    {
        printf("Source file is empty\n");
        close(input_fd);
        return 0;
    }

    /*
     * Map source file into memory.
     */
    source = mmap(
        NULL,

```

```
UW PICO 5.09 File: mmap_copy.c Modified
    {
        perror("open source");
        return 1;
    }

    /*
     * Get file size.
     */
    if (fstat(input_fd, &file_info) == -1)
    {
        perror("fstat");
        close(input_fd);
        return 1;
    }

    size_t size = file_info.st_size;

    if (size == 0)
    {
        printf("Source file is empty\n");
        close(input_fd);
        return 0;
    }

    /*
     * Map source file into memory.
     */
    source = mmap(
        NULL,
        size,
        PROT_READ,
        MAP_PRIVATE,
        input_fd,
        0
    );

    if (source == MAP_FAILED)
    {
        perror("mmap source");
        close(input_fd);
        return 1;
    }

    /*
     * Create destination file.
     */
    output_fd = open(
        argv[2],
        O_RDWR | O_CREAT | O_TRUNC,
        0644
    );

    if (output_fd == -1)
    {
        perror("open destination");
        munmap(source, size);
        close(input_fd);
        return 1;
    }
}
```

```
UW PICO 5.09 File: mmap_copy.c Modified

PROT_READ,
MAP_PRIVATE,
input_fd,
0
);
if (source == MAP_FAILED)
{
    perror("mmap source");
    close(input_fd);
    return 1;
}

/*
 * Create destination file.
 */
output_fd = open(
    argv[2],
    O_RDWR | O_CREAT | O_TRUNC,
    0644
);
if (output_fd == -1)
{
    perror("open destination");
    munmap(source, size);
    close(input_fd);
    return 1;
}

/*
 * Set destination file size.
 */
if (ftruncate(output_fd, size) == -1)
{
    perror("ftruncate");
    munmap(source, size);
    close(input_fd);
    close(output_fd);
    return 1;
}

/*
 * Map destination file into memory.
 */
destination = mmap(
    NULL,
    size,
    PROT_READ | PROT_WRITE,
    MAP_SHARED,
    output_fd,
    0
);
if (destination == MAP_FAILED)
{
    perror("mmap destination");
    munmap(source, size);
}

Get Help WriteOut Read File Prev Pg Cut Text Cur Pos
Exit Justify Where is Next Pg UnCut Text To Spell
```

```
UW PICO 5.09 File: mmap_copy.c Modified

/*
 * Set destination file size.
 */
if (ftruncate(output_fd, size) == -1)
{
    perror("ftruncate");
    munmap(source, size);
    close(input_fd);
    close(output_fd);
    return 1;
}

/*
 * Map destination file into memory.
 */
destination = mmap(
    NULL,
    size,
    PROT_READ | PROT_WRITE,
    MAP_SHARED,
    output_fd,
    0
);
if (destination == MAP_FAILED)
{
    perror("mmap destination");
    munmap(source, size);
    close(input_fd);
    close(output_fd);
    return 1;
}

/*
 * Copy memory directly.
 */
memcpy(destination, source, size);

/*
 * Make sure changes are written.
 */
if (msync(destination, size, MS_SYNC) == -1)
{
    perror("msync");
}

/*
 * Release mappings.
 */
munmap(source, size);
munmap(destination, size);
close(input_fd);
close(output_fd);

printf("File copied using mmap()\n");

Get Help WriteOut Read File Prev Pg Cut Text Cur Pos
Exit Justify Where is Next Pg UnCut Text To Spell
```

```
UW PICO 5.09 File: mmap_copy.c Modified
close(output_fd);
return 1;
}
/*
 * Copy memory directly.
 */
memcpy(destination, source, size);
/*
 * Make sure changes are written.
 */
if (msync(destination, size, MS_SYNC) == -1)
{
    perror("msync");
}
/*
 * Release mappings.
 */
munmap(source, size);
munmap(destination, size);
close(input_fd);
close(output_fd);
printf("File copied using mmap()\n");
return 0;
}

```

Get Help WriteOut Read File Prev Pg Cut Text Cur Pos
Exit Justify Where is Next Pg UnCut Text To Spell

```
vishalkarnati@VISHALS-MacBook-Air week10practical % nano mmap_copy.c
vishalkarnati@VISHALS-MacBook-Air week10practical % gcc mmap_copy.c -o mmap_copy
vishalkarnati@VISHALS-MacBook-Air week10practical % echo "Hello from mmap file I/O" > mmap_input.txt
vishalkarnati@VISHALS-MacBook-Air week10practical % ./mmap_copy mmap_input.txt mmap_output.txt
File copied using mmap()
vishalkarnati@VISHALS-MacBook-Air week10practical % cat mmap_output.txt
Hello from mmap file I/O
vishalkarnati@VISHALS-MacBook-Air week10practical %
```

PART B — Traditional read() / write():

```
UW PICO 5.09 File: readwrite_copy.c Modified
#include <stdio.h>
#include <stdlib.h>
#include <fcntl.h>
#include <unistd.h>

#define BUFFER_SIZE 4096

int main(int argc, char *argv[])
{
    int input_fd;
    int output_fd;

    char buffer[BUFFER_SIZE];

    ssize_t bytes_read;

    if (argc != 3)
    {
        printf("Usage: %s <source> <destination>\n", argv[0]);
        return 1;
    }

    input_fd = open(argv[1], O_RDONLY);
    if (input_fd == -1)
    {
        perror("open source");
        return 1;
    }

    output_fd = open(
        argv[2],
        O_WRONLY | O_CREAT | O_TRUNC,
        0644
    );

    if (output_fd == -1)
    {
        perror("open destination");
        close(input_fd);
        return 1;
    }

    while ((bytes_read = read(
        input_fd,
        buffer,
        BUFFER_SIZE)) > 0)
    {
        ssize_t total_written = 0;
        while (total_written < bytes_read)
        {
            ssize_t written = write(
                output_fd,
                buffer + total_written,
                bytes_read - total_written
            );

            if (written == -1)

```

⌘ Get Help	⌘ WriteOut	⌘ Read File	⌘ Prev Pg	⌘ Cut Text	⌘ Cur Pos
⌘ Exit	⌘ Justify	⌘ Where is	⌘ Next Pg	⌘ UnCut Text	⌘ To Spell

```
UW PIC0 5.09 File: readwrite_copy.c Modified
output_fd = open(
    argv[2],
    O_WRONLY | O_CREAT | O_TRUNC,
    0644
);
if (output_fd == -1)
{
    perror("open destination");
    close(input_fd);
    return 1;
}

while ((bytes_read = read(
    input_fd,
    buffer,
    BUFFER_SIZE)) > 0)
{
    ssize_t total_written = 0;

    while (total_written < bytes_read)
    {
        ssize_t written = write(
            output_fd,
            buffer + total_written,
            bytes_read - total_written
        );

        if (written == -1)
        {
            perror("write");
            close(input_fd);
            close(output_fd);
            return 1;
        }

        total_written += written;
    }

    if (bytes_read == -1)
    {
        perror("read");
    }

    close(input_fd);
    close(output_fd);

    printf("File copied using read/write\n");

    return 0;
}

Get Help      WriteOut      Read File      Prev Pg      Cut Text      Cur Pos
Exit          Justify        Where is       Next Pg      UnCut Text    To Spell
```

```
vishalkarnati@VISHALS-MacBook-Air: week10practical % nano readwrite_copy.c
vishalkarnati@VISHALS-MacBook-Air: week10practical % gcc readwrite_copy.c -o readwrite_copy
vishalkarnati@VISHALS-MacBook-Air: week10practical % ./readwrite_copy mmap_input.txt readwrite_output.txt
File copied using read/write
vishalkarnati@VISHALS-MacBook-Air: week10practical % cat readwrite_output.txt
Hello from mmap file 1/0
vishalkarnati@VISHALS-MacBook-Air: week10practical %
```

